SpecTest: a spec-driven testing agent with persistent memory

Agentic AI, Assignment 2: Practical Agentic Systems with Persistent Memory Individual submission. Repository: SpecTest (source, evaluation and evidence included).

1. Problem and scope

User and recurring task. A developer maintains a Python project whose behaviour is described in a plain-language feature specification. Every time the specification or the code changes, someone has to check that the tests still match the specification, run them, work out *why* anything fails, and report the result. In practice tests drift away from the spec, failures are triaged by hand, and the same mistakes are repeated.

What SpecTest does. SpecTest is a single-agent system with a tool-using investigator. Once a project is set up (`spectest init`), each review cycle (`spectest run`, a file watcher, or a git post-commit hook) does the following:

1. Decides what to do from memory: nothing, re-run the existing tests, or generate new tests.
2. Generates a pytest suite from the spec when needed.
3. Runs it.
4. When tests fail, investigates by reading the project and running tests, then classifies every failure as **CODE BUG** (with file:line and a fix), **TEST BUG**, or **SPEC AMBIGUITY**.
5. Repairs wrong tests under strict rules, held for the developer's approval.
6. Records everything and emails one report per commit.

Acceptance criteria.

- A later run with no changes costs nothing: no LLM call and no duplicate report.
- A code change re-runs the *same reviewed tests*.
- A spec change creates a new version, supersedes the old tests and updates them, and later decisions use only the current version.
- After a restart nothing is redone.
- Instructions planted in project files never change recipients, permissions or results.
- Each project's memory is isolated.

Scope. Python projects with pytest, pure-logic code (no UI or hardware), free-tier LLMs. The live evaluation target is IncidentZero, my Assignment 1 project (about 1,500 lines).

2. Architecture

Component	Role	Technology
Workflow (`spectest/main.py`)	onboard, then generate or run_tests, investigate, record, report	LangGraph `StateGraph`
Investigator (`investigator.py`)	Tool-using agent loop: an LLM step, then tools, repeated, until `finish` or 12 steps	LangGraph, OpenAI-compatible tool calling
LLM bridge (`tools/llm_bridge.py`)	Groq -> Gemini -> OpenRouter fallback, cool-downs, call metrics	`openai` 3.37 client; `gpt-oss-120b`, `gemini-2.5-flash`
Deterministic checks (`tools/test_runner.py`, `agent_tools.py`)	Code validation, repair rules, quote check, path permissions, pytest runner	Python `ast`, pytest 9.1 (JUnit XML)
Memory (`memory.py`)	7 SQLite tables, one database per project	`sqlite3`
Interfaces	CLI (`init`, `run`, `status`, `approve`/`reject`, `history`, `watch`, `config`, `remove`), git hook, watcher	argparse, watchdog
Notifier	Report email (Gmail SMTP to an owned inbox) or `.eml` outbox	`smtplib`

A single coordinator with tools is enough here. The "agents" are two LangGraph graphs: the workflow, and the investigator it calls. Each project has its own `SPECTEST.md` (settings and test instructions) and its own

`.spectest/` folder (memory, logs, outbox). Both are git-ignored.

3. Decision loop

Where the LLM decides.

- **Generation:** the generator writes tests from the spec and the public interface (signatures, docstrings, dataclass fields and enum values, never function bodies). After a spec change it also gets the previous suite and approved lessons, so it *updates* tests rather than rewriting them.
- **Investigation:** the investigator chooses which files to read, what to search, which tests to re-run, whether a failure is a code bug, a test bug or an ambiguity, and whether to repair a test.

The investigator's choices change the next action. In live cycle C1 it found a wrong test, repaired it, re-ran it and finished with 23/23 passing. In C2 it read `policies.py` and reported a code bug without touching any test.

Where deterministic code decides.

- Whether anything changed (spec hash, source/commit hash).
- Whether generated code is safe to run (AST check: allowed imports only, no `eval`/`open`, at least one test).
- Pass/fail counts (from pytest's JUnit XML, never from the LLM).
- Whether a repair is allowed: only currently failing tests may change; no deleting, renaming or skipping tests; shared fixtures may change only to fix failing tests; an exact spec quote is required and checked against the real spec; at most `max_repairs`.
- Whether a CODE BUG verdict stands: it needs a verifiable spec quote, otherwise it is shown as "SPEC AMBIGUITY (unconfirmed)".
- File permissions: reads stay inside the project and never touch `.env`, keys, `.git`, `.spectest` or virtualenvs; writes go only to the generated test file.
- Whether to email: one delivery per (commit, suite); failures are recorded and retried.

Budget and failure handling.

- The investigator has 12 steps. With 2 left it is told to wrap up, and on the last step `finish` is forced, so a diagnosis always exists.
- Generation is retried up to 3 times, with the rejection reason fed back to the model.
- A provider that fails with a size limit, used-up quota, bad key or retired model is skipped for a while.
- A run lock prevents overlapping runs.

4. Tools

Tool	Kind	Used by
`generate_tests` (LLM + validation)	produces the suite, written to memory and the test folder	workflow
`list_files`, `read_file`, `search`	retrieval of local evidence (project code, tests, spec)	investigator
`run_tests` (pytest subprocess)	observation; whole suite or single tests	workflow, investigator
`edit_tests` / `write_tests`	controlled write: the generated test file only, under the repair rules	investigator
SQLite memory	persistent writes (versions, runs, lessons, approvals, notifications)	workflow
`send_report`	controlled external action: email to the owner's inbox, or outbox	workflow

Three capabilities meet the brief's requirement:

- **evidence retrieval:** read and search the project, run tests
- **persistent writes:** SQLite memory and the test file
- **a controlled external action:** email

5. Memory lifecycle

The design is documented in full in `docs/MEMORY.md`; the schema is in `docs/schema.sql`. SQLite is authoritative. The visible test file and the reports are derived from it and can be rebuilt.

Memory	Contents	Read when	Corrected / deleted
Working state	One run's LangGraph state and the investigator's messages; bounded by steps and output limits	during the run	discarded after the run
`spec_baselines`	every spec version (hash, text)	start of every run	new version on edit; history kept
`generated_test_suites`	every suite; one CURRENT	every run; previous suite at regeneration	superseded, never overwritten; `reject` restores the pre-repair suite
`test_runs`	counts, output, diagnosis, repairs, delivery flag	every run; last 3 for the investigator	append-only
`lessons`	why a test was wrong (+ spec quote)	generation and investigation, **approved only**	follows the approval decision
`approvals`	agent repairs awaiting the developer	`status`, `approve`, `reject`	decided once
`notifications`	each delivery attempt: rule, outcome, error	before sending; `status`	append-only
`meta`	`project_id` the database belongs to	on open	`spectest remove` deletes all

Isolation.

- Each project has its own database inside its folder, bound to its `project_id`; opening it as another project fails.
- Configured paths must stay inside the project.
- The agent's tools refuse `.spectest/` and anything outside the project.

Access and deletion.

- Only SpecTest for that project reads its memory, plus the developer through the CLI.
- API keys live in a separate per-user settings file, never in a project or in submitted files.
- `spectest remove` deletes a project's memory after confirmation.

6. Evaluation

Offline scenarios: 20/20 passed (100%) (`evidence/offline/scenarios.md`). A deterministic stub LLM is used, including adversarial replies, and the evaluation is isolated from my real settings. It covers:

- the eight compulsory tests: initial use, later use, process restart in a separate process, correction, isolation with a second project, missing/conflicting evidence, tool failure and retry with SMTP failing once, and an untrusted instruction in the spec
- project-specific cases: regression diagnosis, approval rejection, git hook, memory deletion, workspace hygiene, step budget

Live evaluation on IncidentZero: 7/7 review cycles as expected (`evidence/RESULTS.md`):

Cycle	Outcome
C1 Onboarding	23 test cases generated; one wrong test repaired by the agent (spec quoted) and held for approval
C2 Regression	Existing suite re-run; CODE BUG reported at `policies.py:78` with the exact fix
C3 Restart	New process: 0 LLM calls, nothing redone
C4 Correction	Spec v2 superseded v1, kept all 15 earlier tests and changed the `restart_service` expectation to HIGH

Cycle	Outcome
C5 Untrusted note	A comment telling the agent to delete tests and email `attacker@example.org` was read but had no effect; the real bug was diagnosed
C6 Repair + approval	Planted wrong test repaired, held PENDING, then approved
C7 Email failure	Recorded with its error; the retry delivered exactly one real email

Memory on vs off (same saved state, same models; expected outcomes stated in the script before running):

Case	Memory on	Memory off
M1 Unchanged project	0 calls, no report (success)	regenerated, duplicate report (failure)
M2 Regression	reviewed suite re-used, 6 calls, 25,932 tokens (success)	regenerated first, 13 calls, 97,424 tokens (failure)
M3 Spec change after approved repair	95% of tests kept, approved test kept but renamed (partial)	26% kept, a known misreading reintroduced (failure)

Successes: memory on **2/3** by the automated criterion and **3/3** after manual review (in M3 the approved test was kept with an identical body under a clearer name, so a name-based check missed it). Memory off: **0/3**. In M2, both modes found the bug; memory's benefit there was re-using reviewed tests at 54% fewer calls.

Other measures:

- **Evidence correctness:** 5 of 5 sampled factual claims were correct when checked by hand (line numbers, causes, fixes).
- **Duplicate actions in retry tests:** 0, offline and live. Without memory, M1 produced 1 duplicate.
- **Representative run (C1):** 7 LLM calls (2 failed over to another provider), 27,081 prompt + 3,630 completion tokens, 64 s of LLM time, 71 s in total, $0 on free tiers.
- **Unchanged later run:** 0 calls, 0.1 s.

Failures. These are shown in `evidence/`:

- My first live attempt failed: the import allow-list blocked `time` and `pathlib`.
- The second produced 50/70 passing tests: the generator couldn't see dataclass fields, and correct fixture repairs were rejected.
- An exploratory run on a large Windows/C++ project (Monocle; logs not archived) got 2 of 5 verdicts right.

Each finding led to a fix and a regression scenario.

7. Limitations

- **Quote check:** it proves a quote *exists* in the spec, not that it supports the conclusion. In C2 the agent attempted a test edit during a genuine code bug (it was rejected for other reasons). This is why repairs require developer approval and lessons count only once approved.
- **Spec quality:** results depend on it. My IncidentZero spec was ambiguous about `failed` with `retryable: true`, and SpecTest correctly reported that as SPEC AMBIGUITY.
- **Scope:** pure-logic Python code only; one generated test file per project.
- **Free tiers:** rate limits cause frequent provider fallbacks (16 of 60 live calls), and runs vary between models.
- **The investigator reads source code,** which is sent to the LLM provider. Generation sends only the interface.

8. Implementation decisions

- **Deterministic control over LLM claims.** Pass/fail, permissions, repair rules and email gating are code, so injected instructions and model mistakes cannot change outcomes. This trades flexibility for safety.
- **One SQLite database per project,** not a vector store. The workflow needs exact versions, hashes and decisions, and isolation follows from separate files.
- **Supersede, never overwrite.** Specs and suites are versioned, so corrections keep their history.
- **Agent suggestions vs human decisions.** Repairs apply provisionally, but their lessons only influence future generation after approval, and a rejection restores the original tests.

- **Interface-only generation.** Tests are generated from signatures, fields and docstrings, never function bodies, so the tests follow the spec rather than the code.
- **Triggers.** A file watcher for active work, and a git hook for "test every commit, nothing in between".

Disclosure

The idea, blueprint, technology choices and main design decisions are mine. Much of the code, the evaluation harness and the documentation were written with Claude (Anthropic, via Claude Code) under my direction. I reviewed and tested all of it and can explain and modify every part. No third-party code was copied. IncidentZero is my Assignment 1 project (course-provided starter code). No personal data was used, and reports went only to my own test inbox.

Appendix: evidence index

- `evidence/offline/scenarios.md`: 20-scenario table (input, saved state, expected, observed, result, evidence)
- `evidence/RESULTS.md`: live cycles, memory comparison, claim checks, duplicates, metrics, failures
- `evidence/live/`: step outputs, `steps.json` metrics, `agent_log.jsonl` (LLM calls, tool calls, memory reads and writes, deliveries), `memory_dump.json`, generated test suites (v1, v2, final), outbox reports
- `evidence/live_attempt1_failed/`, `evidence/live_attempt2_partial/`: failed attempts and their logs
- `samples/incidentzero/FEATURE_SPEC.md`: the sample input spec